

Glosario esencial de SQL

Unidad 1 · Lecciones 1 a 6

Oracle Database · esquema OEHR

Propósito de esta guía

Comprender, escribir y revisar las consultas estudiadas en la Unidad 1. Cada tema conecta el propósito del comando con su sintaxis, un ejemplo ejecutable y las precauciones que evitan los errores más frecuentes.

Cómo utilizar esta guía

Esta guía reúne las cláusulas, operadores y funciones trabajados en las seis lecciones de la Unidad 1. Los ejemplos están escritos para Oracle Database y el conjunto de datos OEHR. Puede utilizarse de dos maneras:

- **Lectura progresiva:** seguir las lecciones en orden para construir consultas cada vez más completas.
- **Consulta rápida:** buscar un término en el índice o en el glosario alfabético final.

Cómo leer cada tema. Primero identifique **qué problema resuelve** el comando; luego observe su **sintaxis**; finalmente, ejecute el **ejemplo** y cambie una condición para comprobar cómo varía el resultado. Las cajas naranjas señalan errores que conviene anticipar.

Convención de tablas

Los materiales del repositorio usan nombres como OEHR_EMPLOYEES. Si se trabaja con el esquema estándar HR, la tabla equivalente se escribe HR.EMPLOYEES. Elija una convención y manténgala en toda la consulta; no mezcle ambos nombres.

Tablas del repositorio	Equivalente en el esquema HR
OEHR_EMPLOYEES	HR.EMPLOYEES
OEHR_DEPARTMENTS	HR.DEPARTMENTS
OEHR_JOBS	HR.JOBS
OEHR_LOCATIONS	HR.LOCATIONS
OEHR_COUNTRIES	HR.COUNTRIES
OEHR_REGIONS	HR.REGIONS

Índice

1. Mapa rápido de la unidad	1	6. Lección 4: conversiones, nulos y condiciones	7
2. Anatomía de una consulta	1	6.1. TO_CHAR para fechas	7
2.1. Orden de escritura y orden lógico	1	6.2. TO_CHAR para números	7
3. Lección 1: recuperación básica de datos	2	6.3. TO_NUMBER y TO_DATE	8
3.1. SELECT y FROM	2	6.4. NVL, NVL2, NULLIF y COALESCE	8
3.2. DESCRIBE o DESC	2	6.5. CASE y DECODE	8
3.3. Expresiones aritméticas	2	7. Lección 5: funciones de grupo	9
3.4. Alias de columna	2	7.1. GROUP BY	9
3.5. Concatenación y texto literal	3	7.2. WHERE frente a HAVING	10
3.6. DISTINCT	3	8. Lección 6: consultas multitabla	10
4. Lección 2: filtrar y ordenar	3	8.1. Idea central del JOIN	10
4.1. WHERE y operadores de comparación	3	8.2. Camino entre varias tablas	10
4.2. AND, OR y NOT	3	8.3. ON y USING	11
4.3. BETWEEN y NOT BETWEEN	4	8.4. Tipos de combinación	11
4.4. IN y NOT IN	4	8.5. Autocombinación y unión no igualitaria	11
4.5. LIKE y comodines	4	9. DDL y DML observados brevemente	12
4.6. IS NULL e IS NOT NULL	5	9.1. CREATE TABLE	12
4.7. ORDER BY	5	9.2. INSERT INTO	12
5. Lección 3: funciones de una sola fila	5	10. Errores frecuentes y cómo leerlos	12
5.1. Funciones de texto	5	11. Método recomendado para construir una consulta	13
5.2. Funciones numéricas y DUAL	6	12. Glosario alfabético de consulta rápida	13
5.3. SYSDATE y aritmética de fechas	6	Fuentes internas consultadas	14
5.4. Funciones de fecha	6		

1 Mapa rápido de la unidad

Antes de estudiar comandos aislados, conviene reconocer la progresión de la unidad: primero se recuperan filas, luego se transforman y agrupan y, por último, se relacionan varias tablas.

go se transforman y agrupan y, por último, se relacionan varias tablas.

Lección	Contenidos principales
1	SELECT , FROM , DESCRIBE , expresiones aritméticas, alias, concatenación y DISTINCT .
2	WHERE , operadores de comparación y lógicos, BETWEEN , IN , LIKE , IS NULL y ORDER BY .
3	Funciones de texto, funciones numéricas, tabla DUAL , SYSDATE , aritmética y funciones de fecha.
4	Conversión de tipos, formatos de fecha y número, funciones para nulos y expresiones condicionales.
5	Funciones de grupo, GROUP BY y HAVING .
6	Alias de tabla, productos cartesianos y combinaciones con JOIN , ON y USING .

2 Anatomía de una consulta

Cuando las cláusulas se utilizan juntas, su orden de escritura es estricto:

SELECT	columnas o expresiones
FROM	tabla principal
JOIN	otra tabla
ON	condicion de union
WHERE	condiciones sobre filas
GROUP BY	columnas de agrupacion
HAVING	condiciones sobre grupos
ORDER BY	columnas de ordenamiento;

Regla de lectura. Lea una consulta en dos direcciones: **de arriba abajo** para reconocer lo que está escrito y desde **FROM/JOIN hacia SELECT** para comprender cómo se construye el resultado. Esta segunda lectura explica qué datos existen en cada etapa.

No todas son obligatorias. En una consulta común, **SELECT** indica qué se mostrará y **FROM** indica de dónde se obtendrán los datos. Las otras cláusulas se agregan solo cuando son necesarias. Puede haber varios **JOIN**, cada uno con su propia condición.

Consulta completa: selección, unión, filtro y orden	
<pre>SELECT e.employee_id, e.first_name, d.department_name FROM oe_hr_employees e JOIN oe_hr_departments d ON e.department_id = d.department_id WHERE d.department_name = 'IT' ORDER BY e.last_name;</pre>	

2.1 Orden de escritura y orden lógico

El orden en que Oracle procesa conceptualmente una consulta no es el mismo en que se escribe. Esta diferencia

ayuda a comprender por qué, por ejemplo, **WHERE** no puede usar una función de grupo.

Paso	Escritura	Procesamiento lógico simplificado
1	SELECT	FROM y JOIN
2	FROM	WHERE
3	JOIN ... ON	GROUP BY
4	WHERE	HAVING
5	GROUP BY	SELECT
6	HAVING	ORDER BY
7	ORDER BY	—

3 Lección 1: recuperación básica de datos

Objetivo de aprendizaje. Recuperar columnas de una tabla, calcular expresiones y presentar encabezados y textos comprensibles sin modificar los datos almacenados.

```
SELECT employee_id, first_name, salary
FROM oehr_employees;
```

El asterisco obtiene todas las columnas. Es útil para explorar, pero en reportes conviene escribir únicamente las columnas necesarias.

```
SELECT *
FROM oehr_departments;
```

3.2 DESCRIBE o DESC

Muestra la estructura de una tabla: columnas, tipos de datos y posibilidad de valores nulos. Es un comando del cliente de Oracle, no una consulta **SELECT**.

```
DESCRIBE oehr_locations;
-- Forma abreviada:
DESC oehr_locations;
```

3.3 Expresiones aritméticas

Se pueden usar +, -, * y / dentro de **SELECT**. La multiplicación y la división se evalúan antes que la suma y la resta; los paréntesis permiten controlar el orden.

Ejemplo ejecutable: expresiones aritméticas

```
SELECT employee_id,
       salary,
       salary * 12 AS salario_anual,
       (salary + 57) * 12 AS anual_con_bono_mensual,
       (salary * 12) + 57 AS anual_con_bono_anual
FROM oehr_employees;
```

Si una expresión opera con **NULL**, su resultado también será **NULL**. Más adelante se puede usar **NVL** para sustituirlo.

```
SELECT employee_id AS "COD_EMPLEADO",
       first_name AS "Nombre",
       salary * 12 AS "Sueldo anual"
FROM oehr_employees;
```

3.1 SELECT y FROM

SELECT elige columnas o calcula expresiones. **FROM** señala la tabla que contiene los datos.

Sintaxis: **SELECT** columna_1, columna_2 **FROM** nombre_tabla;

Sintaxis: **DESCRIBE** nombre_tabla;

Sintaxis: **SELECT** expresion_aritmetica **AS** alias **FROM** nombre_tabla;

3.4 Alias de columna

Un alias cambia el encabezado mostrado, pero no modifica la tabla. Las comillas dobles permiten conservar espacios, acentos y mayúsculas exactas.

Sintaxis: expresion **AS** "Encabezado visible"

3.5 Concatenación y texto literal

El operador `||` une valores de texto. Los textos literales se escriben entre comillas simples.

Sintaxis: `expresion_1 || ' texto ' || expresion_2`

```
SELECT employee_id,
       first_name || ' ' || last_name AS nombre_completo,
       'El empleado ' || first_name || ' tiene el cargo ' || job_id AS reporte
FROM oehr_employees;
```

3.6 DISTINCT

Elimina filas duplicadas del resultado según el conjunto de columnas elegido.

Sintaxis: `SELECT DISTINCT columna_1, columna_2 FROM nombre_tabla;`

```
SELECT DISTINCT job_id
FROM oehr_employees;
```

4 Lección 2: filtrar y ordenar

Objetivo de aprendizaje. Restringir filas mediante condiciones, combinar criterios de búsqueda y controlar el orden del resultado final.

4.1 WHERE y operadores de comparación

WHERE conserva únicamente las filas que cumplen una condición.

Sintaxis: `SELECT columnas FROM tabla WHERE condicion;`

Operador	Significado	Operador	Significado
=	igual	<> o !=	distinto
>	mayor que	>=	mayor o igual
<	menor que	<=	menor o igual

```
SELECT employee_id, last_name, salary
FROM oehr_employees
WHERE salary >= 5000;
```

Las cadenas distinguen mayúsculas y minúsculas y se escriben entre comillas simples. Para fechas es preferible

un literal ANSI, pues no depende del formato regional configurado en la sesión.

```
SELECT employee_id, last_name, hire_date
FROM oehr_employees
WHERE hire_date >= DATE '2016-01-01';
```

4.2 AND, OR y NOT

AND exige que ambas condiciones sean verdaderas; **OR** exige al menos una; **NOT** niega una condición. Oracle evalúa

NOT, después **AND** y finalmente **OR**. Los paréntesis hacen explícita la intención.

Sintaxis: `WHERE (condicion_1 OR condicion_2) AND condicion_3`

Ejemplo ejecutable: condiciones combinadas

```
SELECT employee_id, last_name, department_id, salary
FROM oehr_employees
WHERE (department_id = 50 OR department_id = 80)
AND salary >= 5000;
```

4.3 BETWEEN y NOT BETWEEN

Busca dentro o fuera de un rango inclusivo: considera ambos límites.

Sintaxis: WHERE columna [NOT] BETWEEN limite_inferior AND limite_superior

```
SELECT employee_id, last_name, salary
FROM oehr_employees
WHERE salary BETWEEN 3000 AND 5000;
```

```
SELECT employee_id, last_name, hire_date
FROM oehr_employees
WHERE hire_date NOT BETWEEN DATE '2015-01-01' AND DATE '2020-12-31';
```

4.4 IN y NOT IN

Compara una columna con una lista de valores. Es una forma clara de expresar varias comparaciones unidas mediante OR.

Sintaxis: WHERE columna [NOT] IN (valor_1, valor_2, valor_3)

```
SELECT employee_id, last_name, department_id
FROM oehr_employees
WHERE department_id IN (50, 80, 90);
```

```
SELECT employee_id, last_name, manager_id
FROM oehr_employees
WHERE manager_id NOT IN (100, 114, 120, 121);
```

Atención

Si la lista de NOT IN contiene un valor NULL, el resultado puede no devolver filas debido a la lógica de tres valores de SQL. En ese caso se debe revisar el origen de la lista.

4.5 LIKE y comodines

LIKE compara patrones de texto. El símbolo % representa cero o más caracteres y _ representa exactamente un carácter.

Sintaxis: WHERE columna [NOT] LIKE 'patron'

Patrón	Significado
'S%'	comienza con S
'%a'	termina con a
'%an%'	contiene an
'_a%s'	la segunda letra es a y termina con s

```
SELECT employee_id, last_name
FROM oehr_employees
WHERE last_name LIKE 'S%'
OR last_name LIKE '%a';
```

Para buscar literalmente un guion bajo o un porcentaje se puede definir un carácter de escape:

```
SELECT employee_id, job_id
FROM oehr_employees
WHERE job_id LIKE '%SA\_%' ESCAPE '\';
```

Atención

= 'E%' busca exactamente los dos caracteres E%. Para buscar textos que comienzan con E se debe usar LIKE 'E%'. Aun así, el resultado será vacío si los datos unidos no contienen un país que comience con esa letra.

```
SELECT employee_id, last_name, commission_pct
FROM oehr_employees
WHERE commission_pct IS NULL;
```

```
SELECT employee_id, last_name, commission_pct
FROM oehr_employees
WHERE commission_pct IS NOT NULL;
```

4.6 IS NULL e IS NOT NULL

NULL representa un valor ausente o desconocido. No se compara con = ni con <>.

Sintaxis: WHERE columna IS [NOT] NULL

4.7 ORDER BY

Ordena el resultado. **ASC** es ascendente y es el valor por defecto; **DESC** es descendente. Se puede ordenar por una

columna, un alias, una expresión o la posición de una columna del **SELECT**.

Sintaxis: ORDER BY columna_1 [ASC|DESC],
columna_2 [ASC|DESC];

```
SELECT employee_id, last_name, department_id, hire_date
FROM oehr_employees
ORDER BY department_id ASC, hire_date DESC;
```

Aunque **ORDER BY** 2 es válido, usar el nombre o el alias suele ser más claro y resiste mejor los cambios en la lista de columnas.

Cómo interpretar una función. En **FUNCTION(argumento)**, Oracle evalúa el argumento de cada fila y devuelve un valor. Las funciones pueden anidarse: la más interna se ejecuta primero y su resultado alimenta a la siguiente.

5 Lección 3: funciones de una sola fila

Estas funciones producen un resultado por cada fila de entrada.

Objetivo de aprendizaje. Transformar textos, números y fechas fila a fila, sin perder de vista el tipo de dato que recibe y devuelve cada función.

5.1 Funciones de texto

Función	Propósito	Ejemplo
UPPER	Convierte a mayúsculas.	UPPER(last_name)
LOWER	Convierte a minúsculas.	LOWER(email)
INITCAP	Coloca en mayúscula la inicial de cada palabra.	INITCAP(first_name)
CONCAT	Une dos valores. Para más valores debe anidarse o usarse .	CONCAT(first_name, last_name)
LENGTH	Cuenta caracteres.	LENGTH(last_name)
SUBSTR	Extrae una parte desde una posición y con una longitud opcional.	SUBSTR(first_name, 1, 1)
INSTR	Devuelve la posición de una cadena dentro de otra; cero si no existe.	INSTR(last_name, 'a')
TRIM	Elimina caracteres en los extremos; por defecto, espacios.	TRIM(' texto ')
LPAD	Completa a la izquierda hasta una longitud.	LPAD(employee_id, 6, '0')

Función	Propósito	Ejemplo
RPAD	Completa a la derecha hasta una longitud.	RPAD(first_name, 20, '.')
REPLACE	Sustituye apariciones de un texto.	REPLACE(phone_number, '.', '-')

Ejemplo combinado:

Ejemplo ejecutable: funciones de texto

```
SELECT employee_id,
       UPPER(first_name || ' ' || last_name) AS nombre,
       LENGTH(last_name) AS letras,
       INSTR(LOWER(last_name), 'a') AS posicion_a
FROM oehr_employees
WHERE SUBSTR(last_name, -1, 1) = 'n';
```

5.2 Funciones numéricas y DUAL

ROUND(numero, n) redondea; TRUNC(numero, n) corta sin redondear; MOD(a, b) devuelve el resto de dividir *a*

por *b*. DUAL es una tabla especial de Oracle útil para evaluar expresiones sin consultar una tabla de negocio.

Sintaxis: ROUND(numero, decimales) |
TRUNC(numero, decimales) | MOD(a, b)

```
SELECT ROUND(45.5674, 2) AS redondeado,
       TRUNC(45.5674, 2) AS truncado,
       MOD(54, 3) AS resto
FROM dual;
```

El separador decimal escrito en SQL suele ser el punto. La forma correcta de MOD recibe dos argumentos: MOD(54, 3).

5.3 SYSDATE y aritmética de fechas

SYSDATE devuelve la fecha y hora actuales del servidor Oracle. No lleva paréntesis.

Sintaxis: SELECT SYSDATE FROM DUAL;

```
SELECT SYSDATE
FROM dual;
```

En Oracle, sumar o restar un número a una fecha suma o resta días. La diferencia entre dos fechas produce una cantidad de días.

```
SELECT SYSDATE + 1 AS mañana,
       SYSDATE - 7 AS hace_una_semana
FROM dual;
```

```
SELECT employee_id,
       hire_date,
       TRUNC(SYSDATE - hire_date) AS dias_trabajados,
       TRUNC((SYSDATE - hire_date) / 7) AS semanas_trabajadas
FROM oehr_employees;
```

Atención

Una columna DATE también almacena hora. Por eso `fecha = SYSDATE` rara vez encuentra una fila: tendrían que coincidir fecha y hora. Para com-

parar solo el día puede usarse `TRUNC(fecha) = TRUNC(SYSDATE)`.

5.4 Funciones de fecha

Función	Descripción
MONTHS_BETWEEN(f1,f2)	Cantidad de meses entre dos fechas; devuelve un número.
ADD_MONTHS(fecha,n)	Suma o resta meses a una fecha.
NEXT_DAY(fecha,día)	Devuelve la siguiente ocurrencia del día indicado. El idioma del nombre depende de NLS_DATE_LANGUAGE.
LAST_DAY(fecha)	Devuelve el último día del mes de la fecha.
ROUND(fecha,formato)	Redondea una fecha a la unidad indicada.
TRUNC(fecha,formato)	Trunca una fecha a la unidad indicada; sin formato quita la hora.

```
SELECT employee_id,
       hire_date,
       MONTHS_BETWEEN(SYSDATE, hire_date) AS meses,
       ADD_MONTHS(hire_date, 6) AS revision,
       NEXT_DAY(hire_date, 'MONDAY') AS lunes_siguiente,
       LAST_DAY(hire_date) AS fin_de_mes
FROM oehr_employees;
```

6 Lección 4: conversiones, nulos y condiciones

Objetivo de aprendizaje. Convertir datos de forma explícita, controlar la presentación de fechas y números, tratar valores nulos y producir resultados condicionales.

6.1 TO_CHAR para fechas

Convierte una fecha a texto utilizando un modelo de formato.

Sintaxis: TO_CHAR(fecha, 'modelo_de_formato')

Formato	Significado	Formato	Significado
DD	día del mes	MM	mes numérico
MONTH	nombre del mes	YYYY	año de cuatro dígitos
DAY	nombre del día	HH24	hora de 00 a 23
MI	minutos	SS	segundos
MON	mes abreviado	DY	día abreviado
YEAR	año escrito en letras	AM / PM	indicador de meridiano

```
SELECT TO_CHAR(SYSDATE, 'DD/MM/YYYY HH24:MI:SS') AS fecha_hora,
       TO_CHAR(SYSDATE, 'DAY DD "DE" MONTH "DE" YYYY') AS fecha_larga
FROM dual;
```

El modificador FM elimina espacios y ceros de relleno agregados por algunos formatos. El texto escrito entre comillas dobles dentro del modelo se reproduce literalmente.

```
SELECT TO_CHAR(SYSDATE, 'FMDAY DD "DE" MONTH "DE" YYYY')
FROM dual;
```

6.2 TO_CHAR para números

Convierte un número en texto con el formato de presentación solicitado.

Sintaxis: TO_CHAR(numero, 'modelo_de_formato')

En los modelos numéricos, 9 reserva una posición opcional, 0 obliga a mostrar un cero, , y . representan separadores y \$ imprime el símbolo indicado. La configuración NLS de la sesión puede influir en algunos separadores y símbolos monetarios.

```
SELECT employee_id,
       TO_CHAR(salary, '$999,999.99') AS sueldo_formateado
FROM oehr_employees;
```

El resultado de `TO_CHAR` es texto. Sirve para presentar datos, no para continuar cálculos numéricos.

6.3 TO_NUMBER y TO_DATE

`TO_NUMBER` convierte texto a número y `TO_DATE` convierte texto a fecha usando el formato indicado.

Sintaxis: `TO_NUMBER(texto, 'formato')` |
`TO_DATE(texto, 'formato')`

```
SELECT TO_NUMBER('2026', '9999') AS anio,
       TO_DATE('11/09/2026', 'DD/MM/YYYY') AS fecha
FROM dual;
```

Cuando una fecha es fija, el literal ANSI suele ser más claro:

```
SELECT DATE '2026-09-11' AS fecha
FROM dual;
```

Se deben evitar conversiones implícitas como `hire_date > '01/01/2016'`, porque dependen de la configuración regional y pueden fallar o interpretarse de forma distinta.

6.4 NVL, NVL2, NULLIF y COALESCE

Estas funciones permiten decidir qué valor devolver cuando un dato es nulo o cuando dos expresiones resultan iguales.

Función	Resultado
<code>NVL(valor, reemplazo)</code>	Devuelve el reemplazo cuando el valor es NULL ; en caso contrario devuelve el valor original.
<code>NVL2(v, no_nulo, nulo)</code>	Elige entre dos resultados según el valor sea o no nulo.
<code>NULLIF(a, b)</code>	Devuelve NULL si <i>a</i> y <i>b</i> son iguales; si no, devuelve <i>a</i> .
<code>COALESCE(a, b, c, ...)</code>	Devuelve el primer argumento que no sea NULL .

```
SELECT employee_id,
       NVL(commission_pct, 0) AS comision,
       NVL2(commission_pct, 'CON COMISION', 'SIN COMISION') AS estado
FROM oehr_employees;
```

```
SELECT NULLIF(10, 10) AS resultado_nulo,
       COALESCE(NULL, NULL, 'Primer valor disponible') AS resultado
FROM dual;
```

Los posibles resultados de `NVL` y las alternativas de `NVL2` deben ser de tipos compatibles.

6.5 CASE y DECODE

CASE expresa condiciones de manera legible. **DECODE** es una función propia de Oracle para comparaciones por

igualdad; cuando la lógica es más compleja suele preferirse **CASE**.

Sintaxis: `CASE WHEN condicion THEN resultado`
`ELSE alternativa END`

Ejemplo ejecutable: expresión CASE

```
SELECT employee_id,
       salary,
       CASE
         WHEN salary >= 10000 THEN 'ALTO'
         WHEN salary >= 5000  THEN 'MEDIO'
         ELSE 'BAJO'
       END AS nivel_sueldo
FROM oehr_employees;
```

```
SELECT employee_id,
       DECODE(department_id,
              50, 'Shipping',
              80, 'Sales',
              'Otro') AS area
FROM oehr_employees;
```

7 Lección 5: funciones de grupo

Objetivo de aprendizaje. Resumir conjuntos de filas mediante totales, promedios, extremos y conteos; después, formar y filtrar grupos correctamente.

Las funciones de grupo producen un resultado por conjunto de filas e ignoran valores **NULL**, excepto **COUNT(*)**, que cuenta filas.

Función	Propósito
AVG (expresion)	promedio
SUM (expresion)	suma
MIN (expresion)	mínimo
MAX (expresion)	máximo
COUNT (expresion)	cantidad de valores no nulos
COUNT (*)	cantidad de filas
COUNT (DISTINCT expresion)	cantidad de valores distintos no nulos

```
SELECT ROUND(AVG(salary), 0) AS promedio,
       SUM(salary) AS total,
       MIN(hire_date) AS primera_contratacion,
       MAX(hire_date) AS ultima_contratacion,
       COUNT(*) AS empleados
FROM oehr_employees;
```

7.1 GROUP BY

Sintaxis: **SELECT** columna, **FUNCION_GRUPO**(...) **FROM** tabla **GROUP BY** columna;

Forma grupos. Toda expresión del **SELECT** que no esté dentro de una función de grupo debe aparecer en **GROUP BY**.

```
SELECT department_id,
       job_id,
       TO_CHAR(SUM(salary), '$999,999.99') AS total_sueldos
FROM oehr_employees
GROUP BY department_id, job_id
ORDER BY department_id;
```

7.2 WHERE frente a HAVING

WHERE filtra filas antes de agrupar. **HAVING** filtra los grupos ya calculados y puede usar funciones de grupo.

Sintaxis: WHERE condicion_de_fila GROUP BY columnas HAVING condicion_de_grupo

Ejemplo ejecutable: agrupación y filtro de grupos

```
SELECT department_id,
       MAX(salary) AS sueldo_maximo
FROM   oehr_employees
WHERE  department_id BETWEEN 50 AND 90
GROUP BY department_id
HAVING MAX(salary) > 10000
ORDER BY department_id DESC;
```

No es válido escribir **WHERE** `MAX(salary) > 10000` porque el máximo todavía no existe en la etapa donde se ejecuta **WHERE**.

8 Lección 6: consultas multitabla

Objetivo de aprendizaje. Relacionar tablas mediante sus claves, calificar columnas con alias y elegir el tipo de combinación que conserva las filas necesarias para el reporte.

```
SELECT e.employee_id,
       e.first_name,
       d.department_name
FROM   oehr_employees e
JOIN   oehr_departments d
      ON e.department_id = d.department_id;
```

Los alias `e` y `d` hacen la consulta más corta y evitan ambigüedad cuando dos tablas contienen columnas con el mismo nombre.

8.1 Idea central del JOIN

Un **JOIN** no modifica ni mezcla físicamente las tablas. Construye un resultado temporal relacionando filas. La condición de **ON** suele comparar una clave foránea con la clave primaria correspondiente.

Sintaxis: FROM tabla_a a JOIN tabla_b b ON a.clave_foranea = b.clave_primaria

8.2 Camino entre varias tablas

Para obtener el país y la región de un empleado se sigue la relación del modelo:

EMPLOYEES → DEPARTMENTS → LOCATIONS → COUNTRIES → REGIONS

Ejemplo ejecutable: recorrido entre cinco tablas

```
SELECT e.employee_id AS "ID empleado",
       e.first_name AS "Nombre",
       c.country_name AS "País",
       l.city AS "Ciudad",
       r.region_name AS "Región"
FROM   oehr_employees e
JOIN   oehr_departments d
      ON e.department_id = d.department_id
JOIN   oehr_locations l
      ON d.location_id = l.location_id
JOIN   oehr_countries c
      ON l.country_id = c.country_id
JOIN   oehr_regions r
      ON c.region_id = r.region_id
ORDER BY e.last_name;
```

Para conocer qué países existen realmente en ese resultado antes de filtrar:

```
SELECT DISTINCT c.country_name
FROM oehr_employees e
JOIN oehr_departments d
    ON e.department_id = d.department_id
JOIN oehr_locations l
    ON d.location_id = l.location_id
JOIN oehr_countries c
    ON l.country_id = c.country_id
ORDER BY c.country_name;
```

8.3 ON y USING

ON permite escribir cualquier condición de unión y es la forma más explícita. **USING** se puede usar cuando la columna relacionada tiene el mismo nombre en ambas tablas.

Sintaxis: tabla_a JOIN tabla_b USING
(columna_comun)

```
SELECT l.city, d.department_name
FROM oehr_locations l
JOIN oehr_departments d USING (location_id);
```

La columna escrita dentro de **USING** no debe llevar alias dentro de la lista: se escribe **USING** (location_id), no **USING** (l.location_id).

8.4 Tipos de combinación

Combinación	Comportamiento
INNER JOIN / JOIN	Devuelve solo filas con coincidencia en ambas tablas.
LEFT JOIN	Conserva todas las filas de la tabla izquierda; coloca NULL cuando no hay coincidencia a la derecha.
RIGHT JOIN	Conserva todas las filas de la tabla derecha.
FULL OUTER JOIN	Conserva las filas de ambos lados, coincidan o no.
CROSS JOIN	Genera todas las combinaciones posibles; es un producto cartesiano.
NATURAL JOIN	Une automáticamente columnas del mismo nombre. Debe usarse con cuidado porque una nueva columna homónima puede cambiar el resultado.

Ejemplo para mostrar empleados aunque no tengan departamento:

```
SELECT e.employee_id,
       e.last_name,
       d.department_name
FROM oehr_employees e
LEFT JOIN oehr_departments d
    ON e.department_id = d.department_id;
```

8.5 Autocombinación y unión no igualitaria

Una autocombinación usa dos alias de la misma tabla. Aquí se relaciona cada empleado con su jefe:

```
SELECT e.last_name AS empleado,
       j.last_name AS jefe
FROM oehr_employees e
LEFT JOIN oehr_employees j
    ON e.manager_id = j.employee_id;
```

Una unión no igualitaria usa una condición diferente de =, por ejemplo para ubicar un salario dentro de un rango:

```
SELECT e.last_name, e.salary, g.grade_level
FROM oehr_employees e
JOIN job_grades g
ON e.salary BETWEEN g.lowest_sal AND g.highest_sal;
```

La tabla JOB_GRADES es ilustrativa y debe existir para ejecutar el ejemplo.

Atención

Si se enumeran tablas sin una condición correcta, se genera un producto cartesiano: cada fila de una tabla se combina con cada fila de la otra. La consulta puede ejecutarse, pero sus datos no representan la relación real.

ron observadas, aunque su estudio detallado corresponde a otros bloques de la unidad.

Alcance. En esta unidad se observaron CREATE TABLE e INSERT INTO. Se incluyen para reconocer su propósito y su forma básica; el estudio completo de DDL y DML se realiza en unidades posteriores.

9.1 CREATE TABLE

Crea una tabla y define sus columnas y tipos.

Sintaxis: CREATE TABLE nombre (columna tipo_de_dato);

9 DDL y DML observados brevemente

En la parte 2 de la lección 3 aparecen dos instrucciones fuera de la consulta de datos. Se registran aquí porque fue-

```
CREATE TABLE ejemplo (
  rut VARCHAR2(12)
);
```

9.2 INSERT INTO

Agrega una fila. Los valores de texto deben ir entre comillas simples.

```
INSERT INTO ejemplo (rut)
VALUES ('13.572.343-2');
```

Sintaxis: INSERT INTO tabla (columnas) VALUES (valores);

10 Errores frecuentes y cómo leerlos

Un mensaje de error no es solo un obstáculo: indica en qué parte de la consulta debe revisarse la sintaxis o la

lógica. La siguiente tabla traduce los casos más habituales a una acción concreta.

Situación	Causa común	Corrección
No aparecen filas con LIKE 'E%'	No hay valores relacionados que comiencen con E.	Quitar temporalmente el filtro y consultar DISTINCT.
invalid identifier	Nombre de columna incorrecto, por ejemplo DEPARTMENTS_ID.	Revisar con DESCRIBE; la columna es DEPARTMENT_ID.
column ambiguously defined	Dos tablas poseen una columna con el mismo nombre.	Anteponer el alias: e.department_id.
Demasiadas filas o datos incoherentes	Falta una condición de unión o se unieron columnas equivocadas.	Revisar cada JOIN ... ON siguiendo claves primarias y foráneas.
Una comparación con NULL no funciona	Se escribió = NULL.	Usar IS NULL o IS NOT NULL.
Una fecha fija cambia de interpretación	Se confió en el formato regional.	Usar DATE 'YYYY-MM-DD' o TO_DATE con formato explícito.

Situación	Causa común	Corrección
Error de agrupación	Una columna del SELECT no está agrupada ni dentro de una función de grupo.	Agregarla a GROUP BY o aplicar una función de grupo.

11 Método recomendado para construir una consulta

Construir la consulta por capas permite localizar errores rápidamente y evita escribir una sentencia extensa antes de comprobar sus relaciones básicas.

1. Escribir con palabras qué datos debe mostrar el reporte.
2. Identificar en qué tabla se encuentra cada columna.
3. Elegir la tabla principal y escribir **FROM**.

4. Trazar el camino entre tablas y agregar cada **JOIN ... ON**.
5. Ejecutar sin filtros y revisar una muestra del resultado.
6. Agregar **WHERE** y comprobar los valores existentes con **DISTINCT** cuando sea necesario.
7. Agregar agrupaciones si el reporte usa totales o promedios.
8. Ordenar al final con **ORDER BY**.
9. Reemplazar **SELECT *** por las columnas exactas del reporte.

12 Glosario alfabético de consulta rápida

Esta sección concentra los comandos y conceptos de la unidad. Para aprender un tema por primera vez, consulte la explicación y el ejemplo de su lección; use este glosario para recordar su propósito durante la práctica.

ADD_MONTHS Suma meses a una fecha.

AND Exige que dos condiciones sean verdaderas.

AS Introduce un alias de columna; suele ser opcional.

ASC / DESC Define orden ascendente o descendente.

AVG Calcula un promedio.

BETWEEN Comprueba un rango inclusivo.

CASE Devuelve resultados diferentes según condiciones.

COALESCE Devuelve el primer valor no nulo.

CONCAT Une dos cadenas.

COUNT Cuenta filas o valores no nulos.

CREATE TABLE Crea una tabla.

CROSS JOIN Genera todas las combinaciones posibles entre dos tablas.

DATE (literal ANSI) Representa una fecha fija como **DATE 'YYYY-MM-DD'**, sin depender del formato de sesión.

DECODE Realiza comparaciones condicionales por igualdad en Oracle.

DESCRIBE / DESC Muestra la estructura de una tabla en el cliente Oracle.

DISTINCT Elimina duplicados del resultado.

DUAL Tabla especial de Oracle para evaluar expresiones.

ESCAPE Define el carácter que permite buscar % o _ literalmente en un patrón de **LIKE**.

FULL OUTER JOIN Conserva filas coincidentes y no coincidentes de ambas tablas.

FROM Define el origen de datos.

GROUP BY Forma grupos para cálculos agregados.

HAVING Filtra grupos.

IN Compara con una lista de valores.

INITCAP Convierte la inicial de cada palabra a mayúscula.

INNER JOIN Devuelve únicamente las filas que coinciden en ambas tablas.

INSERT INTO Agrega una fila a una tabla.

INSTR Busca la posición de un texto dentro de otro.

IS NULL Comprueba si un valor es nulo.

JOIN Combina filas relacionadas de dos tablas.

LAST_DAY Devuelve el último día del mes.

LEFT JOIN Conserva todas las filas de la tabla izquierda.

LENGTH Cuenta caracteres.

LIKE Compara un texto con un patrón.

LPAD / RPAD Completa un valor por la izquierda o la derecha.

LOWER / UPPER Convierte texto a minúsculas o mayúsculas.

MAX / MIN Obtiene el valor máximo o mínimo.

MOD Devuelve el resto de una división.

MONTHS_BETWEEN Calcula meses entre dos fechas.

NATURAL JOIN Une columnas homónimas automáticamente; requiere especial cuidado.

NEXT_DAY Devuelve el próximo día de semana indicado.

NOT Niega una condición.

NOT BETWEEN / IN / LIKE Niega el rango, la lista o el patrón correspondiente.

NULL Representa ausencia o desconocimiento de un valor.

NULLIF Devuelve nulo cuando dos expresiones son iguales.

NVL / NVL2 Sustituye o clasifica valores nulos.

ON Declara la condición de un **JOIN**.

OR Exige que al menos una condición sea verdadera.

ORDER BY Ordena el resultado final.

REPLACE Sustituye apariciones de un texto dentro de otro.

RIGHT JOIN Conserva todas las filas de la tabla derecha.

ROUND / TRUNC Redondea o trunca números y fechas.	SYSDATE Fecha y hora actuales del servidor Oracle.	TRIM Elimina espacios u otros caracteres en los extremos de un texto.
SELECT Elige columnas y expresiones que se mostrarán.	TO_CHAR Convierte números o fechas a texto con formato.	USING Abrevia una unión cuando la columna se llama igual en ambas tablas.
SUBSTR Extrae una parte de una cadena.	TO_DATE Convierte texto a fecha con formato.	WHERE Filtra filas antes de agrupar.
SUM Suma los valores de un grupo.	TO_NUMBER Convierte texto a número.	

Fuentes internas consultadas

Archivos SQL: U1/codigos/leccion-01.sql, leccion-02.sql, leccion-03.sql, leccion-03-parte-02.sql, leccion-04.sql, leccion-04-parte-02.sql, leccion-05.sql y leccion-06.sql. **Presentaciones:** lecciones 1 a 6 disponibles en U1/presentaciones/.

Antes de entregar una consulta. Compruebe que las tablas y columnas existen, que cada JOIN sigue una relación válida, que los textos y fechas usan literales correctos, que toda columna no agregada figura en GROUP BY y que los alias describen con claridad el resultado.